TECH CUP - Engineering Assignment

¡Bienvenidos al Sprint #4 del desafío técnico de TechCup! Estas semanas continuamos con el desarrollo progresivo del proyecto. Durante este sprint se introducirán nuevas implicaciones que deben ser analizadas e integradas en la solución actual . El objetivo de este ejercicio es simular un entorno real de ingeniería, donde los sistemas evolucionan sprint tras sprint y el equipo debe adaptar su solución de acuerdo al feedback obtenido.

Cómo adicional este sprint tiene una mayor duración que los sprints pasados, iniciando desde la exposición que cada equipo realizó del sprint #3, hasta el día 12 de Abril.

CONSIDERACIONES DEL TRABAJO EN EQUIPO

Para el desarrollo del proyecto deberán cumplir con las siguientes dinámicas de trabajo:

1. Planning:

Cada equipo deberá realizar una reunión inicial de planificación (**Planning**) al inicio de cada sprint, con una duración definida por el equipo según su necesidad.

En esta reunión se debe:

- Definir el **alcance del sprint** (qué se va a desarrollar).
- Priorizar y distribuir las **tareas entre los integrantes**.
- Estimar el esfuerzo de las actividades.
- Identificar posibles **riesgos o bloqueos iniciales**.
- Alinear expectativas sobre entregables y tiempos.

NOTA: Deben dejar evidencia de la reunión que incluya:

a. Resumen de lo discutido.
b. Integrantes que participaron.
c. Tareas asignadas y responsables.
d. Decisiones tomadas.

2. Dailies:

Cada equipo deberá realizar **reuniones diarias (Daily)** de aproximadamente **15 minutos**. En estas reuniones cada integrante debe responder:

- Qué hice ayer con respecto a mis asignaciones
- Qué haré hoy
- Qué bloqueos tengo

NOTA: Si no pueden realizar la reunión virtual, pueden hacerla presencial, pero deben dejar evidencia escrita que incluya:

- resumen de lo discutido por todos los integrantes
- integrantes que asistieron
- decisiones tomadas

3. Comunicación del Equipo:

El equipo debe tener un canal de comunicación activo para coordinar el proyecto, puede ser mediante:

- **Microsoft Teams**
- **WhatsApp**

El docente debe estar **incluido en este canal** para poder observar el proceso de colaboración del equipo.

4. Organización del Equipo:

Cada sprint deberán compartir:

- **Organigrama del equipo:** Debe incluir sus roles asignados, este organigrama puede cambiar de sprint a sprint.
- **Disponibilidad del equipo:** Cada integrante debe indicar: horarios disponibles y posibles restricciones de tiempo según sus actividades académicas o laborales

Esto permitirá planear correctamente el trabajo del sprint.

ACTIVIDADES DEL SPRINT #4

Durante este sprint deberán realizar las siguientes actividades

1. IDENTIFICACIÓN Y SOLUCIÓN DE LA DEUDA TÉCNICA SPRINT #3:

Antes de continuar con el desarrollo del sistema, el equipo debe revisar el estado actual del proyecto junto con los comentarios de feedback que se presentaron en la sesión de revisión por parte del profesor.

Entre las actividades se incluyen:

- Revisar los requerimientos establecidos y actualizarlos con el feedback recomendado - recuerde la planeación sobre funcionalidades no sobre generalidades.
- Revisar el análisis de requerimientos y actualizarlo con los comentarios realizados.
- Identificación de inconsistencias en las definiciones de los artefactos entregados:
    - Mockups del sistema - Correcciones de forma y de estructura, navegación, flujos de error y flujos por roles.

- Manual de identidad visual - Correcciones de logotipos, colores utilizados y explicación técnica como soporte.
- Repositorio Back End - Readme técnico corregido con forma y estructura técnica de definiciones y explicaciones.
- Repositorio Back End - Estructura MVC con el scaffolding adecuado y manejo por separado de modelo y entidades.
- Repositorio Front End - Generación y correcciones en el Readme tecnico
- Diagramas del sistema - Correcciones:
    - Diagrama de Contexto
    - Diagrama de Clases - Junto a los patrones a implementar
    - Diagrama de componentes General
    - Diagrama de componentes Específico
    - Diagramas de Secuencia por operaciones (Implementadas y por Implementar)
    - Diagrama de Entidad - Relación.

- Detección de deuda técnica - Puntos faltantes si no fueron desarrollados
    - Funcionalidades implementadas en BackEnd
    - Seguridad y manejo de JWT y OAuth2 para autenticación.
    - Manejo de Roles y permisos por funcionalidades.
    - Implementación de la persistencia con JPA para las entidades relacionales del sistema.
    - Set de pruebas y su estructura por funcionalidad
    - Implementación de Análisis estático de pruebas y Cobertura de código
    - Organización de la documentación con Swagger

2. REFINAMIENTO DEL BACKLOG:

Una vez identificada la deuda técnica, el equipo deberá organizar el trabajo del proyecto dentro de JIRA:

A. Product Backlog:

a. Dentro de cada una de las ÉPICAS establecidas, genere las tareas necesarias asociadas a la deuda técnica presentada en el Sprint #3
NOTA: Cada deuda técnica identificada debe registrarse como una feature o tarea independiente dentro del backlog atada a la épica correspondiente.

b. Genere un **feature** asociado a la **dockerización del sistema**, cuyo objetivo sea permitir la estandarización del entorno de ejecución, facilitar el despliegue y asegurar la portabilidad de la aplicación.

El feature debe estar correctamente **fragmentado en tareas técnicas**, contemplando como mínimo:

- Definición del **Dockerfile** para la aplicación (únicamente backend).
- Configuración de variables de entorno.

- Exclusión de credenciales sensibles del código fuente.
- Creación de un archivo **docker-compose.yml** para orquestar los servicios necesarios (ej: aplicación, base de datos local para desarrollo).
- Configuración de redes y volúmenes persistentes.
- Integración de variables de conexión a base de datos mediante **environment variables**.
- Validación del correcto levantamiento del entorno en local.
- Documentación de uso en el README (cómo construir la imagen y levantar los contenedores).
- Estrategia para desacoplar la base de datos local vs. remota (Azure).

c. Genere un **feature** asociado a la **implementación de un pipeline de CI/CD**, cuyo objetivo sea automatizar la integración continua, pruebas y despliegue del sistema.

Debe tener en cuenta que la solución se implementará utilizando:

- **GitHub Actions**
- **Azure (App Services, Azure Database for PostgreSQL, Azure Container Registry u otros recursos equivalentes)**

El feature debe estar correctamente **fragmentado en tareas técnicas**, incluyendo como mínimo:

I. Pipeline de Integración Continua (CI)

- Configuración del workflow en **GitHub Actions**.
- Definición de triggers (ej: push a ramas dependiendo el ambiente - develop (QA) , main/master (PROD)).
- Instalación de dependencias del proyecto.
- Ejecución de pruebas unitarias.
- Ejecución del análisis estático.
- Configuración de variables de entorno por ambiente (base de datos de QA y PROD).
- Generación de artefactos (builds, imágenes Docker si aplica).
- Publicación de resultados de ejecución.
- Generación del despliegue con la URL por ambiente

II. Gestión de Secrets y Seguridad

- Configuración de **GitHub Secrets** para almacenar credenciales sensibles:
  - Usuario de base de datos
  - Password de base de datos
  - Host y puerto
  - Connection string
- Uso de secrets dentro de los workflows de **GitHub Actions**.

- Configuración de **Azure Key Vault (opcional pero recomendado)** para centralizar secretos.
- Inyección de variables sensibles en tiempo de ejecución (no hardcodeadas).
- Rotación básica de credenciales (conceptual).

III. Conectividad con Base de Datos (Azure PostgreSQL)

- Configuración de instancia en **Azure Database for PostgreSQL**.
- Definición de reglas de firewall para permitir acceso desde:
  - Servicios de Azure (App Service / AKS)
  - Pipelines de GitHub Actions
- Configuración de cadena de conexión (connection string).
- Integración de la aplicación con la base de datos mediante variables de entorno.
- Validación de conexión desde entornos:
  - QA
  - Producción
- Manejo de migraciones de base de datos (opcional pero recomendado).

IV. Pipeline de Despliegue a QA

- Configuración de entorno en **Azure para QA**.
- Despliegue automático al ambiente de pruebas.
- Inyección de **secrets de QA** (credenciales de base de datos QA).
- Validación del despliegue (health checks básicos).
- Verificación de conectividad con PostgreSQL en Azure.

V. Pipeline de Producción

- Definición de workflow independiente o condicionado (ej: merge a `main/master`).
- Uso de **aprobaciones manuales** antes de producción por lo menos 3 miembros del equipo.
- Construcción de artefactos productivos.
- Publicación de imágenes en **Azure Container Registry (ACR)**.
- Despliegue en ambiente productivo (App Service).
- Inyección de **secrets de producción** (base de datos productiva).
- Validación post-deploy (health checks + conexión a BD).

VI. **Gestión del Pipeline**

- Definición clara de **steps del pipeline**:
  - Build
  - Test
  - Analysis
  - Security (manejo de secrets)
  - Deploy QA
  - Deploy Producción
- Manejo de fallos.
- Versionamiento de artefactos e imágenes Docker.

VII. Recuerde actualizar el Readme de su aplicación con los links URL de los despliegues en QA y PROD.

d. Para la feature de seguridad recuerde:
   i. Agregue encode y decode a los datos sensibles (contraseñas) que almacena en base de datos, se sugiere que use base64 para esto.

e. Para las feature de arquitectura relacionado con diagramación incluya:
   i. Diagrama de despliegue de la aplicación - Enfocado en el detalle de cómo interactúan los componentes en el despliegue de la aplicación.

3. REFINAMIENTO DEL SET DE PRUEBAS:

A partir de lo visto en clase con respecto a la cobertura y análisis de código deberán agregar a la definición de los escenarios de prueba para cada funcionalidad **TANTO LAS YA IMPLEMENTADAS como LAS QUE VAN A IMPLEMENTAR**:

- Evidencias de los registros de cobertura a nivel de Líneas, Ramas, Métodos y Clases con Jacoco
- Evidencia de la ejecución con SonarQube del análisis estático de código
  - Si los calificativos son menores a B tiene que realizar las tareas correspondientes para mejorar y subir sus puntuaciones.

Estos deben agregarse a JIRA como evidencia dentro de cada feature finalizado.

4. IMPLEMENTACIÓN DE PANTALLAS FRONT END:

Una vez su mockup esté completo con el flujo básico de funcionalidades, el flujo de errores y todas las pantallas contemplen el buen uso de UX/UI para la estructuración y a partir de lo que se va a ver en clase en semana 11 deben iniciar con la implementación de las pantallas.

Genere las siguientes tareas para front end:

1. Implementación del scaffolding/estructura de carpetas del proyecto de la siguiente manera:

```
src/
 ├── assets/            # imágenes, íconos, fuentes
 ├── components/        # componentes reutilizables
 ├── features/          # módulos por funcionalidad (domain-driven)
 ├── pages/             # pantallas principales
 ├── hooks/             # custom hooks
 ├── services/          # capa de consumo de APIs
 ├── store/             # manejo de estado (Redux, Zustand, Context)
 ├── types/             # tipados globales (interfaces, DTOs)
 ├── utils/             # funciones helper
 ├── routes/            # configuración de rutas
 └── styles/            # estilos globales o tema
```

2. Configure linter y formateo para su proyecto:
    - ESLint
    - Prettier
3. Genere una matriz de trazabilidad de funcionalidades pero principalmente enfocada en el front y la generación de los flujos de pantallas.
4. Por cada funcionalidad definida en el mockup, se debe crear una **rama independiente tipo `feature/*`**.
    a. Cada feature debe incluir las siguientes tareas:
        - Creación de la(s) pantalla(s) correspondiente(s) en `pages/`.
        - Implementación de componentes reutilizables en `components/`.
        - Separación de lógica y presentación (hooks personalizados si aplica).
        - Tipado de props y modelos usando TypeScript.
        - Implementación de estados visuales:
            - Loading
            - Error
            - Empty states
        - Implementación de validaciones de formularios (ej: login, registro, pago).
        - Manejo de navegación entre pantallas.
        - Aplicación consistente de estilos (según diseño UX/UI definido en sus mockups).
        - Pruebas básicas funcionales.

5. Para el tema de la conectividad tenga en cuenta la conectividad con axios genere un feature enfocado en las siguientes tareas a realizar:
        - Instalación y configuración de **Axios**.
        - Creación de una instancia base (`apiClient`) con:
            - `baseURL` desde variables de entorno
            - Timeouts

    - Headers por defecto
- Interceptor de request:
    - Inyección de tokens
- Interceptor de response:
    - Manejo centralizado de errores
    - Normalización de respuestas
- Creación de servicios por dominio en `services/`:
    - **Ejemplo:** `auth.service.ts`
- Definición de métodos por endpoint:
    - GET (consultas)
    - POST (creación)
    - PUT/PATCH (actualización)
    - DELETE (eliminación)
- Definición de interfaces para:
    - Request DTOs
    - Response DTOs
- Uso de tipado fuerte en cada llamada API.
- Consumo de servicios desde hooks o componentes.
- Manejo de estados:
    - Loading
    - Error
    - Success
- Manejo de errores de red vs errores de negocio.

5. IMPLEMENTACIÓN DE CÓDIGO - BACKEND:

Durante este sprint deberán continuar con el desarrollo a nivel de código de sus funcionalidades, utilizando una arquitectura MVC.

Deben implementar:

- Por funcionalidad escogida y tareas a realizar este sprint, deberán crear todo el flujo desde el controlador hasta la creación de los servicios, utilidades y validadores.
- Pruebas de acuerdo al set de pruebas que surge para esa funcionalidad que se va a trabajar.
- Manejo de errores.
- Implementación de la persistencia.

6. DOCUMENTO DE ARQUITECTURA - BACK END:

- Siguiendo el estándar que se vio en las sesiones teóricas, es necesario que desarrollen el documento de arquitectura del backend, recuerde tener una portada, un índice de los puntos que componen el documento, buena organización del trabajo realizado y buena redacción.

- Se puede apoyar del uso de la I.A para generar una mejor redacción de lo que plantean, sin embargo la I.A no debe reemplazar las ideas base que propongan.

7. CONSTRUCCIÓN DEL SPRINT BACKLOG SPRINT #4:

A. Después de organizar correctamente la planeación del trabajo, Identifique cuáles tareas son más prioritarias, cuáles tareas tienen correlación directa, que tarea puede bloquear a otra o es derivada de otra.
B. El líder de la semana anterior tiene que cerrar el Sprint #3, resumiendo en el cierre que se logró y que quedó como deuda técnica para el Sprint #4.
C. Del Backlog Seleccione las actividades que planean realizar y desarrollar en este sprint, recuerde que tiene que ser concordante con la matriz de trazabilidad que realizaron en Sprint #1 y que han corregido en los demás sprint - Adicional recuerde la cantidad de semanas de trabajo faltante, distribuyan bien la carga laboral.
D. Arrastre cada una de esas tareas al Sprint Backlog del Sprint #4.
E. Una vez seleccionadas, realicen la correcta estimación de las tareas, recordando usar Planning Poker para la definición, Fibonacci para los puntajes y recordando que la carga máxima por persona. - Recuerde el balance entre todos los integrantes del equipo.
F. Los ingenieros Devops son los encargados de las tareas de CI/CD y los ingenieros de front End deben estar enfocados en la creación de las pantallas - Recuerde que deben estar alineados con las responsabilidades.
G. Finalmente deberán Iniciar el Sprint en JIRA y definir el objetivo para este SPRINT #4.

NOTA: Recuerde que para el día 12 de Abril el 100% de la implementación del código de backend con las funcionalidades necesarias, seguridad y persistencia debe estar terminado.

Para ese mismo día deben tener un avance con respecto a la implementación de CI/CD en su proyecto, teniendo mínimo el link de QA expuesto en su despliegue en azure.

Para ese mismo día deben tener un avance del 50% de las pantallas a implementar en el front end, teniendo conectividad con el backend por medio de axios.

8. SPRINT REVIEW:

Al finalizar el sprint deberán realizar una presentación del trabajo realizado, la cual será expuesta en la sesión de laboratorio por cada líder técnico. La presentación debe incluir:

- **Miembros del equipo y que rol tuvo cada uno esta semana**
- **Objetivo del sprint:** Qué buscaba lograr el equipo durante esta semana.
- **Distribución de tareas:** Cómo se repartió el trabajo dentro del equipo, Como quedo la matriz de trazabilidad y el sprint backlog para el SPRINT #4.

- **Métricas del Sprint:** Puntos asignados a cada integrante, Puntos logrados por cada integrante y puntos totales del sprint
- Demo funcional: Demostración del sistema funcionando y ejecutado con spring boot - Correcciones realizadas y Que lograron avanzar este sprint

9. **SPRINT RETROSPECTIVE: Retroalimentación del Equipo**

   Finalmente el equipo debe reflexionar sobre el proceso respondiendo:

   - ¿Qué hicieron bien?
   - ¿Qué hicieron mal?
   - ¿Qué mejorarían en el siguiente sprint?
   - ¿A qué se comprometen para el próximo sprint?

ENTREGABLES:

Dentro del espacio de moodle deben enviar:

- **Link de su repositorio BackEnd**
- **Link de su repositorio FrontEnd**
- **Link de sus mockups con Figma**
- **Documento de Análisis de Requerimientos**
- **Documento de Manual de Identidad**
- **Documento de Arquitectura - Back End**
- **Presentación de Canva o Powerpoint de la retrospectiva y review**